



Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 20	
Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	
Title of Assignment:	Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations
Problem Statement & Learning Objective	Problem Statement Develop a Java application to establish connectivity with a database and perform INSERT, UPDATE, and DELETE operations. Learning Objectives To perform database modification operations using JDBC.
Theory	JDBC allows Java applications to perform database manipulation using SQL Data Manipulation Language (DML) statements such as INSERT, UPDATE, and DELETE. The INSERT statement adds new records into database tables, UPDATE modifies existing records, and DELETE removes unwanted records. These operations are executed using the Statement or PreparedStatement interface. PreparedStatement is generally preferred because it supports parameterized queries, improves performance, and prevents SQL injection attacks. Proper transaction management ensures that database modifications are performed safely and consistently. These operations form the basis of almost every database-driven application such as banking systems, hospital management systems, and student information systems.
Program Code	<pre>import java.sql.*;</pre>

```

public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college2",
                "root",
                "P@resh123!"
            );

            Statement stmt = con.createStatement();

            // INSERT
            stmt.executeUpdate(
                "INSERT INTO student VALUES (426, 'Akshit',
                'Cyber Security', 78)"
            );
            System.out.println("Record inserted successfully.");

            // UPDATE
            stmt.executeUpdate(
                "UPDATE student SET marks = 86 WHERE roll_no =
                426"
            );
            System.out.println("Record updated successfully.");

            // DELETE
            stmt.executeUpdate(
                "DELETE FROM student WHERE roll_no = 426"
            );
            System.out.println("Record deleted successfully.");

            con.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

Output Screenshots

```

PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 20> & 'c:\Program Files\Eclipse Adoptiu
m\jdk-24.0.2.12-hotspot\bin\java.exe' '@C:\Users\Nilima\AppData\Local\Temp\cp_5jd2eq6wzt8io
vghl3k6or1si.argfile' 'Main'
Record inserted successfully.
Record updated successfully.
Record deleted successfully.
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 20>

```

**Exercise Qns
and Code**

1. Create a Java application to manage **employee records** using CRUD operations.

```
import java.sql.*;

public class Exp1 {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/company2",
                "root",
                "P@resh123!"
            );

            Statement stmt = con.createStatement();

            // CREATE - Insert employee
            stmt.executeUpdate(
                "INSERT INTO employee VALUES (247, 'Akshit',
'Finance', 68500)"
            );
            System.out.println("Employee inserted
successfully.");

            // UPDATE - Update salary
            stmt.executeUpdate(
                "UPDATE employee SET salary = 73200 WHERE emp_id
= 247"
            );
            System.out.println("Employee updated
successfully.");

            // READ - Display employee
            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM employee"
            );

            System.out.println("\nEmployee Records:");

            while (rs.next()) {
                System.out.println(
                    "ID: " + rs.getInt("emp_id") +
                    ", Name: " + rs.getString("name") +
                    ", Department: " +
rs.getString("department") +
                    ", Salary: " + rs.getDouble("salary")
                );
            }

            // DELETE - Delete employee
            stmt.executeUpdate(
```

```

        "DELETE FROM employee WHERE emp_id = 247"
    );
    System.out.println("Employee deleted
successfully.");

```

```

        con.close();
    }
    catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

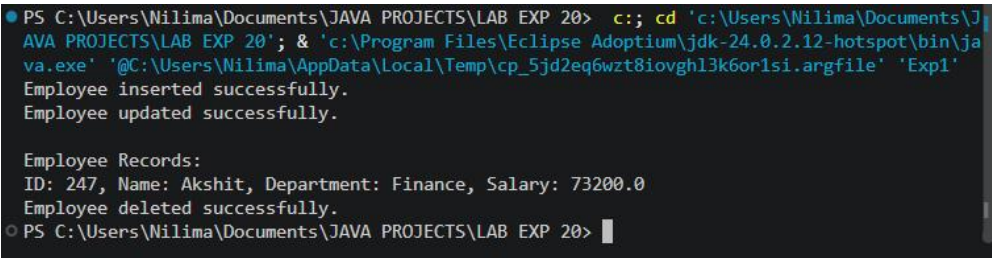
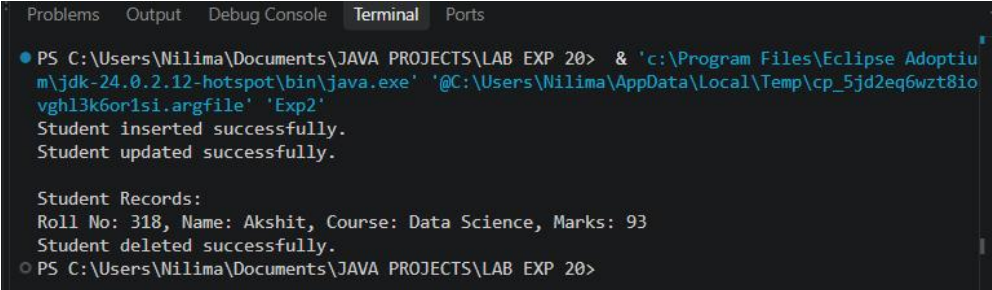
2. Create a Java application to manage **student records** using **CRUD operations**. Perform Create, Read, Update, and Delete operations on student details such as roll number, name, course, and marks using a database.

```

import java.sql.*;

public class Exp2 {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college2",
                "root",
                "P@resh123!"
            );
            Statement stmt = con.createStatement();
            // CREATE - Insert student
            stmt.executeUpdate(
                "INSERT INTO student VALUES (318, 'Akshit',
'Data Science', 87)"
            );
            System.out.println("Student inserted
successfully.");
            // UPDATE - Update marks
            stmt.executeUpdate(
                "UPDATE student SET marks = 93 WHERE roll_no =
318"
            );
            System.out.println("Student updated successfully.");
            // READ - Display students
            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM student"
            );
            System.out.println("\nStudent Records:");
            while (rs.next()) {
                System.out.println(
                    "Roll No: " + rs.getInt("roll_no") +
                    ", Name: " + rs.getString("name") +
                    ", Course: " + rs.getString("course") +
                    ", Marks: " + rs.getInt("marks")
                );
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

	<pre>); } // DELETE - Delete student stmt.executeUpdate("DELETE FROM student WHERE roll_no = 318"); System.out.println("Student deleted successfully."); con.close(); } catch (Exception e) { System.out.println("Error: " + e.getMessage()); } } } </pre>
Output Screenshots	Exp1:  Exp2: 
Conclusion	This experiment completes relational database management using PreparedStatement for Data Manipulation Language tasks. It highlights executing parameter-bound SQL queries via executeUpdate() to maintain dynamic application data securely, preventing SQL injection vulnerabilities while managing database transaction outcomes.